Newton School of Technology

Lecture: 0001

Course Intro & Boolean Algebra

Where It All Begins

The journey from a single transistor to the GPUs that train modern AI — starting with 1s and 0s.

CSA222: Modern Computer Architecture

Welcome to Computer Architecture

The course that reveals what's happening beneath every line of code you've ever written.

THE PROMISE

By the end, you'll understand — from the ground up — how a computer actually runs a program.

Not 'a CPU executes instructions' hand-waving. The real thing: transistors → gates → adders → registers → datapath → instructions → your Python.

You'll be able to trace A[i] = B[i] + C[i] through every single layer.

WHY IT MATTERS TO YOU

Performance: why some code is 100× faster.

AI/ML engineering: GPUs and TPUs are architecture — you can't optimise what you don't understand.

Interviews & systems work: this is the layer that separates people who use computers from people who understand them.

What Is Computer Architecture?

The design of how hardware is organised to run software — the bridge between the two.

ARCHITECTURE

The 'what' — the interface

- What instructions a CPU offers.
- How registers & memory are seen.
- The programmer's contract.
- e.g. 'this CPU has an ADD instruction'.

ORGANISATION

The 'how' — the implementation

- How the ADD is actually built.
- Pipelines, caches, clock speed.
- Hidden from the programmer.
- e.g. 'ADD takes 1 cycle via this adder'.

This course covers both — starting from the gates up, so 'how' and 'what' make sense together.

Why Should You Care?

Four concrete reasons this is one of the most useful courses you'll take.

PERFORMANCE

Understanding caches & pipelines is why one loop runs 100× faster than another.

AI / ML

GPUs, TPUs, and tensor cores ARE architecture. Module 6 connects directly to modern AI.

SYSTEMS DEPTH

OS, compilers, databases — every systems field rests on the hardware model you learn here.

INTERVIEWS

Architecture questions separate candidates. This is signal that you understand computing deeply.

1. Course Logistics

How this course runs: 12 weeks, six modules, labs and contests — and how you'll be graded.

How the Course Runs

12 weeks · 24 lectures · 2 lectures + 2 labs each week.

24 lectures

Two 60-minute lectures per week across 12 weeks.

Hands-on labs

Two labs weekly: build real circuits in Verilog, simulat, write assembly in MIPS .

6 modules

From digital logic up to multicore & AI accelerators — each 2 weeks, 4 lectures.

Contests & project

Regular contests keep skills sharp; a project ties concepts into something you build.

The Six-Module Journey

Each module builds on the last — from a single gate to an AI accelerator.

M1 Wk 1–2

Digital Foundations

Gates, Boolean algebra, combinational blocks, number systems.

M2 Wk 3–4

Sequential Circuits & Memory

Flip-flops, registers, counters, FSMs, register files.

M3 Wk 5–6

Computer Organization & ISA

Instruction sets, assembly, how programs become machine code.

M4 Wk 7–8

CPU Datapath & Pipelining

Build a working CPU; make it fast with pipelining.

M5 Wk 9–10

Memory Hierarchy & Cache

Why memory is slow, and how caches hide it.

M6 Wk 11–12

Parallelism & Modern

Multicore, GPUs, TPUs — the hardware behind modern AI.

How You're Graded

Weighted to reward consistent effort, not just exam-cramming.

100% total — spread across the semester

40%

End-sem exam

Comprehensive final.

20%

Contests

Regular skill checks — keep pace.

20%

Mid-sem exam

Halfway checkpoint.

10%

Projects

Build something real.

10%

Assignments

Weekly practice.

Tools We'll Use

Real industry-adjacent tools — you'll build and simulate, not just read.

Verilog + Simulation

Describe hardware in a real HDL and watch signals in a waveform viewer. Modules 1–2.

MIPS

A MIPS assembler & simulator — write and run assembly by hand. Module 3.

Hardware boards

Real hardware to see logic and timing in the physical world.

How to Succeed Here

This course rewards building intuition, not memorising facts.

1

Draw everything

Circuits, truth tables, timing — get it on paper. Architecture is a visual subject.

2

Do the labs seriously

You truly learn this by building and simulating, not just watching slides.

3

Connect the layers

Always ask: how does this connect up to code, and down to transistors?

4

Ask in lectures

Cold-calls and questions are normal here — confusion voiced early saves hours later.

5

Keep pace with contests

They're spaced so you can't cram. Steady effort beats a panic before finals.

6

Think in trade-offs

There's rarely one 'right' design — area vs speed vs power. That mindset is the goal.

2. The Abstraction Stack

From a switch made of silicon to a neural network — every layer built on the one below it.

The Abstraction Stack

Each layer hides the one below it — you can work at one level trusting the rest.

↑ higher = more abstract

This course climbs the middle: gates → datapath → ISA — the heart of the stack.

One Line of Code, Every Layer

Watch A[i] = B[i] + C[i] descend from Python to silicon.

Python	`A[i] = B[i] + C[i]`	You write one obvious line.
↓		
Machine code	`lw · lw · add · sw`	Compiler turns it into CPU instructions.
↓		
Datapath	`regfile → ALU → regfile`	Registers feed an adder; result written back.
↓		
Gates & transistors	`XOR, AND, carry logic`	The adder is gates; gates are transistors.

Same computation, four languages of description. By this course's end, you'll speak all four.

Where We Start: The Bottom

Today we begin at the very foundation — the logic of 1s and 0s.

EVERYTHING RESTS ON THIS

A transistor is just a switch: on or off. 1 or 0.

Combine switches and you get logic gates. Combine gates and you get arithmetic, memory, and eventually a CPU.

The mathematics that governs how 1s and 0s combine is Boolean algebra — and that's exactly where we start today.

TODAY'S DESTINATION

1. **Boolean values**
   what 1 and 0 really mean
2. **The core gates**
   AND, OR, NOT, XOR
3. **Truth tables**
   the complete behaviour of a gate
4. **Boolean laws**
   the rules for simplifying logic

3. Boolean Basics

The algebra of true and false — invented by George Boole in 1847, and the language every computer speaks.

Boolean Values: Just 1 and 0

Two values, infinite possibility. Everything digital is built from this binary choice.

1

TRUE · HIGH · ON

voltage present (~3.3V or 5V)

0

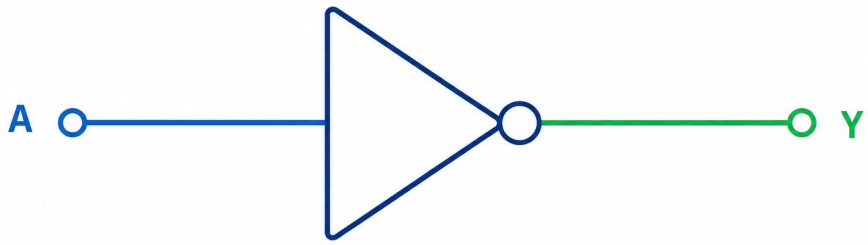
FALSE · LOW · OFF

no voltage (~0V, ground)

The big idea: represent everything — numbers, text, images, instructions — as patterns of these two values. Reliability comes from only needing to tell 'voltage' from 'no voltage'.

The NOT Gate (Inverter)

One input, one output — it simply flips the value. 1 becomes 0, 0 becomes 1.



`Y = A'` (also written NOT A, or Ā)

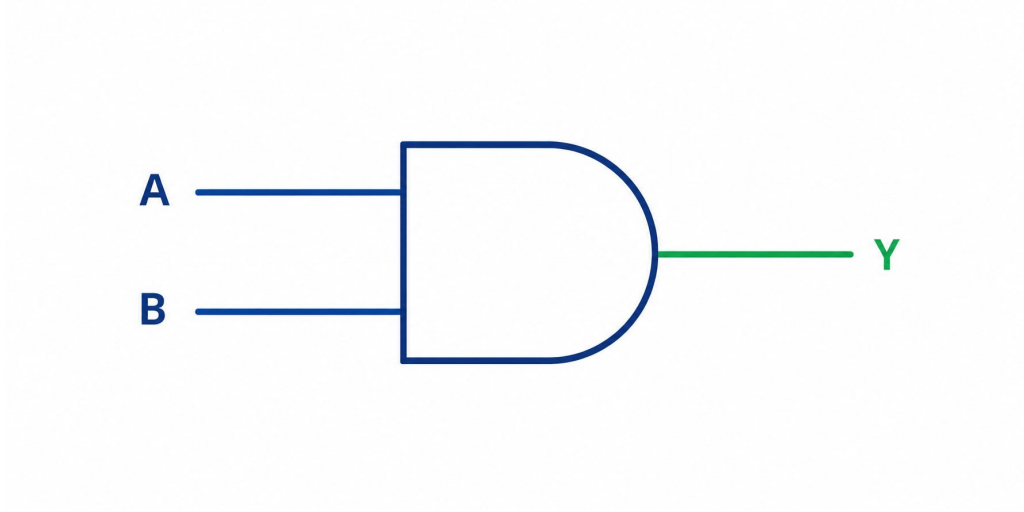
TRUTH TABLE

A	Y
0	1
1	0

The simplest gate — but essential. Inversion is what lets logic express 'not this'.

The AND Gate

Output is 1 only when BOTH inputs are 1. Think: a series of two switches.



`Y = A · B` (also written A AND B)

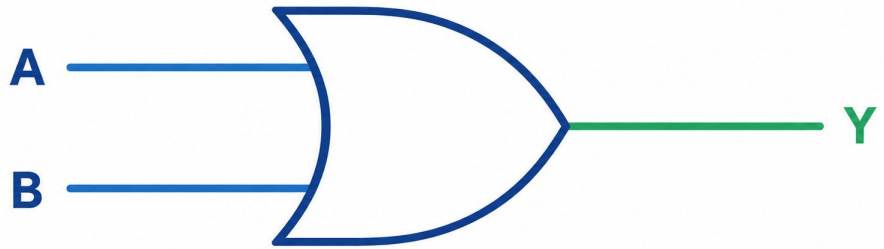
TRUTH TABLE

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

Only the last row — both 1 — gives output 1. 'All conditions must hold.'

The OR Gate

Output is 1 when AT LEAST ONE input is 1. Think: two switches in parallel.



`Y = A + B` (also written A OR B)

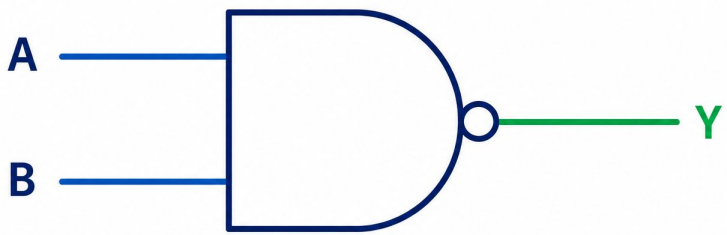
TRUTH TABLE

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

Only the first row — both 0 — gives 0. 'Any one condition is enough.'

The NAND Gate

Output is 0 when both the inputs are 1.



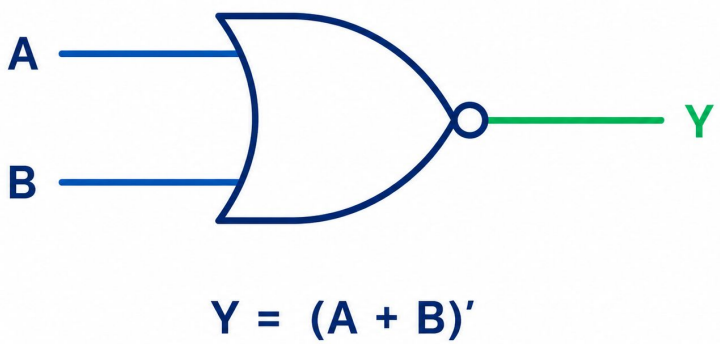
$$Y = (AB)'$$

TRUTH TABLE

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

The NOR Gate

Output is 1 when both the inputs are 0.



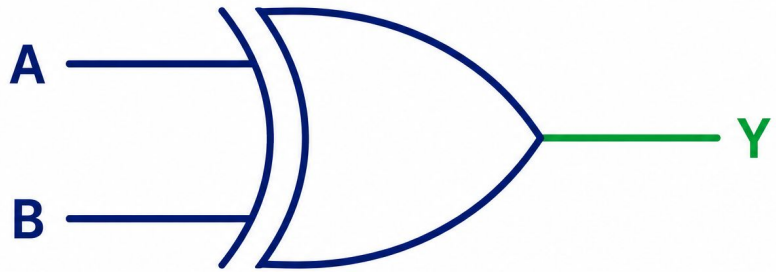
$$Y = (A + B)'$$

TRUTH TABLE

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

The XOR Gate

Output is 1 when the inputs DIFFER. The 'exclusive or' — one or the other, not both.



$$Y = A \oplus B$$

$$Y = A'B + AB'$$

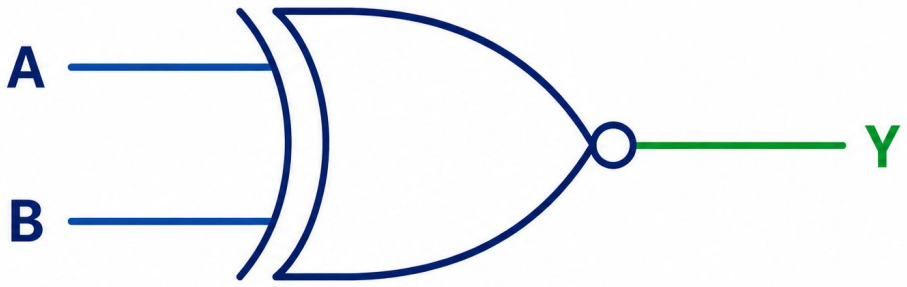
TRUTH TABLE

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

Same inputs → 0, different → 1. XOR is the heart of the adder you'll build in L3.

The XNOR Gate

Output is 1 when the inputs are same.



$$Y = A \odot B \quad (A \text{ XNOR } B)$$

$$Y = A'B' + AB$$

TRUTH TABLE

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

Same inputs then output is → 1, different → 0.

The Four Gates at a Glance

Same two inputs A, B — four different rules for the output.

AND

A·B

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

OR

A+B

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

XOR

A⊕B

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

NAND

(A·B)'

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

Notice: NAND is just AND with the output flipped. That tiny change makes it something special — next slide.

NAND & NOR: The Universal Gates

A teaser for L2 — from just one gate type, you can build everything.

ONE GATE TO RULE THEM ALL

NAND = NOT-AND. NOR = NOT-OR.

Remarkable fact: using ONLY NAND gates, you can build AND, OR, NOT — and therefore ANY logic circuit at all.

NAND is called 'functionally complete' or universal. NOR is too.

`e.g.  NOT A  =  A NAND A`

Feed A into both inputs of a NAND → you get NOT A.

WHY IT'S HUGE

Chip factories can optimise for making ONE gate type extremely well.

NAND is cheap, fast, and dense in silicon — so real chips use enormous numbers of them.

We'll prove NAND's universality properly in L2, then use it to minimise real circuits.

Quick Check

Show of hands — no notes.

Q. For inputs A = 1, B = 0 — which gate outputs 1: AND, OR, or both?

A. Only AND

B. Only OR

✓ *OR needs just one 1*

C. Both

D. Neither

AND needs BOTH inputs 1 → gives 0 here. OR needs at least one 1 → gives 1. Different inputs, so XOR would also be 1.

4. Boolean Laws

The algebra: simple rules that let us simplify logic — fewer gates, cheaper and faster circuits.

Identity & Null Laws

What happens when you combine a variable with a constant 0 or 1.

IDENTITY LAWS

combining with the 'do-nothing' value

A + 0 = A

OR with 0 changes nothing

A · 1 = A

AND with 1 changes nothing

NULL (DOMINANCE) LAWS

combining with the 'dominant' value

A + 1 = 1

OR with 1 is always 1

A · 0 = 0

AND with 0 is always 0

These four are the 'multiply by 1, add 0' of Boolean algebra — obvious once you see them.

Idempotent & Complement Laws

Combining a variable with itself, or with its own inverse.

IDEMPOTENT LAWS

a variable combined with itself

A + A = A

OR-ing A with itself is just A

A · A = A

AND-ing A with itself is just A

COMPLEMENT LAWS

a variable and its inverse A'

A + A' = 1

something OR not-itself is always true

A · A' = 0

something AND not-itself is always false

Complement laws are the workhorses of simplification — they make whole terms vanish to 0 or 1.

The Absorption Law

A tidier rule that 'absorbs' a redundant term — a preview of real simplification.

THE LAW

$$A + A \cdot B = A$$

Read it: once you already have A, adding 'A AND anything' tells you nothing new.

If A is true, the whole expression is true regardless of B. If A is false, A·B is false too. Either way, it's just A.

Its partner: $A \cdot (1 + B) = A$.

WHY SIMPLIFY?

Fewer terms = fewer gates.

Fewer gates = smaller chip area, less power, and often faster circuits.

This is the entire economic reason Boolean algebra matters in hardware — and it's what L2's K-maps automate.

De Morgan's Laws (Intuitive Preview)

How to push a NOT through an AND or an OR — the most-used identity in all of logic.

$$(A \cdot B)' = A' + B'$$

NOT(A AND B) = (NOT A) OR (NOT B)

"Not both" means "at least one is missing."

$$(A + B)' = A' \cdot B'$$

NOT(A OR B) = (NOT A) AND (NOT B)

"Neither" means "this off AND that off."

The rule of thumb: break the bar, flip the sign. A NOT over the whole expression flips AND↔OR and inverts each variable. We'll prove it in L2.

Worked Example: Simplify

Put the laws to work — turn a messy expression into a tiny one.

SIMPLIFY: Y = A·B + A·B'

1	Y = A·B + A·B'	*the starting expression*
2	Y = A·(B + B')	*factor out A (distributive law)*
3	Y = A·(1)	*because B + B' = 1 (complement law)*
4	Y = A	*because A·1 = A (identity law)*

THE PAYOFF

Started with 2 AND gates + 1 OR gate + a NOT.

Ended with just a wire — Y = A.

Four gates eliminated. That's real silicon saved, from four lines of algebra. This is the power you're learning.

Common Mistakes to Avoid

Trip-ups that catch nearly everyone at first.

Reading '+' as addition

In Boolean algebra + means OR, not arithmetic add. A + 1 = 1, not 2.

Confusing OR with XOR

OR is 1 for '1,1' too. XOR is 0 for '1,1'. Watch that last row.

Forgetting NAND ≠ AND

NAND is AND with the output INVERTED. The bubble on the symbol matters.

Over-trusting intuition

Always verify a simplification with a truth table until the laws are second nature.

1 and 0 aren't 'true/false only'

They're also HIGH/LOW voltages, ON/OFF switches — same idea, physical meaning.

Key Takeaways

Five sentences — the foundation the whole course is built on.

01 Computer architecture is the bridge from software down to transistors — this course climbs the middle of that stack.

02 Everything digital reduces to two values, 1 and 0, physically just voltage present or absent.

03 Four core gates — AND (·), OR (+), NOT ('), XOR (⊕) — are fully described by their truth tables.

04 NAND and NOR are universal: from one gate type you can build any circuit — the reason real chips use them.

05 Boolean laws (identity, null, complement, absorption, De Morgan) let us simplify logic into fewer, cheaper gates.

Practice & Homework

Build the habits: truth tables, simplification, and drawing.

01

Truth Table Drill

Write the full truth tables for XOR, NAND, and NOR from memory. Verify against the gate definitions.

02

Simplify

Simplify Y = A·(A + B) and Y = A + A'·B using the laws. Name each law you use.

03

Draw It

Draw the gate-level circuit for Y = A + B·C. Then build its truth table.

EXIT TICKET

1. What is the output of an XOR gate when both inputs are 1?
2. Simplify A·(A + B) and name the law.
3. Where in the abstraction stack does 'the ISA' sit relative to logic gates and Python?

Rapid-Fire Recap

Three questions — shout the answers.

Q1 AND(1, 0) equals… `0`

Q2 XOR outputs 1 when the inputs are… `different`

Q3 A + 1 equals… `1`

All three instant? You've got the foundation — L2 dives into truth tables, K-maps, and universal gates.

END OF L01 · WELCOME ABOARD

From two values, everything.

NEXT — L02

Logic Minimisation & Universal Gates — truth tables to circuits, K-maps for simplification, and proving NAND can build anything.

Thanks for attending!